설비 고장을 예측하는 AI 데이터 분석

문자열 다루기

치환·분리·결합

posco

설비 고장을 예측하는 AI 데이터 분석

문자열 다루기

문자열의 이해와 생성 Part 1

문자열 만들기

자료형과 형변환

인덱싱과 슬라이싱 Part 2

인덱싱

슬라이싱

연산·검색과 정리 메서드 Part 3

정리 메서드

f-string과 텍스트 정리 Part 4

f-string

치환·분리·결합

posco

replace() - 단어 바꾸기

replace() - 특정 글자·단어를 다른 것으로 바꾸기

괄호에 바꿀 것, 새 것 순으로 - replace('A', 'B')

등장하는 걸 모두 한 번에 바꿈

결과는 변수에 다시 받기

replace로 공백·기호 제거

바꿀 대상을 **빈 값('')**으로 바꾸면 그 글자가 사라짐

replace(' ', '')는 중간 공백까지 모두 제거

전화번호 하이픈도 replace('-', '')로 제거

strip이 못 하던 중간 공백을 해결

replace 제거 - 코드 예시

빈 값으로 바꿔 공백·기호 제거

CODE

```
text = '정 상 가 동'
text = text.replace(' ', '')   # 공백 제거
print(text)                    # 정상가동
phone = '010-1234-5678'.replace('-', '')
print(phone)                   # 01012345678
```

replace 여러 번 연결

replace를 **체이닝**으로 이어 여러 종류 차례로 바꾸기

replace('-', '').replace(' ', '')처럼

앞 결과에 다음 replace를 바로 적용

왼쪽부터 차례로 정리됨

데이터 정리에서 replace 활용

replace의 두 용도 - **표기 통일**과 **기호 제거**

'fault' 'FAULT'를 모두 '고장'으로 통일

'3,000'의 쉼표를 제거해 int로 변환 가능

지저분한 글자를 분석 가능한 형태로 다듬기

실습 1. 단어 바꾸기

목표	replace() 로 문자열의 특정 단어를 다른 단어로 바꾸기
단계	① '설비 정상 가동' 저장 → ② '정상'을 '점검'으로 바꿔 출력
예상 결과	설비 점검 가동

실습 1. 단어 바꾸기 정답

CODE

```
s = "설비 정상 가동"
print(s.replace("정상", "점검"))   # 설비 점검 가동
```

여러 조각을 담는 리스트 맛보기

리스트 - 여러 값을 순서대로 담는 그릇

대괄호 안에 값들을 쉼표로 나열 (사과, 배, 감)

번호로 조각 하나를 꺼냄 (0부터, 문자열과 동일)

자세한 건 다음 단원 - 지금은 split 결과 이해용

split() - 구분자로 나누기

split() - 정해진 **구분자**로 문자열을 여러 조각으로

구분자가 나오는 자리마다 잘라 리스트로

슬라이싱은 위치로 자르기, split은 구분자로 나누기

조각들은 리스트에 순서대로 담김

split 기본 - 공백 기준

괄호를 비우면 **공백**을 기준으로 나눔

'에스프레소 아메리카노 카페라테'.split()은 세 단어

공백이 여러 칸이어도 알아서 단어만 나눔

문장을 단어로 쪼갤 때 사용

실습 2. 공백 기준으로 나누기

목표	split() 으로 공백을 기준으로 문장을 단어 리스트로 나누기
단계	① 'PUMP A 03' 저장 → ② 공백 기준 분리
예상 결과	['PUMP', 'A', '03']

실습 2. 공백 기준으로 나누기 정답

CODE

```
s = "PUMP A 03"
print(s.split())    # ['PUMP', 'A', '03']  (공백 기준)
```

split(',') - 쉼표로 나누기

괄호에 구분자를 넣으면 그 구분자로 나눔

'사과,배,감'.split(',')은 사과·배·감 세 조각

구분자는 사라지고 사이 값만 남음

하이픈·빗금 등 데이터에 맞는 구분자 선택

split 결과는 리스트

split 결과는 **리스트** - 번호로 원하는 조각 꺼내기

'2025-01-15'.split('-')은 2025·01·15

0번은 연도, 1번은 월, 2번은 일

슬라이싱으로 위치 세던 것보다 훨씬 편함

split 활용 - CSV 한 줄 나누기

CSV 한 줄을 split(',')으로 나눠 **각 값 꺼내기**

'1,EQP-001,정상,25.3'을 네 조각으로

0번 사이클, 1번 코드, 2번 상태, 3번 측정값

측정값은 float로 바꿔 계산에 활용

실습 3. 쉼표 기준으로 나누기

목표	split(',') 으로 쉼표를 기준으로 나누기
단계	① 'a,b,c,d' 저장 → ② ','을 기준으로 분리
예상 결과	['a', 'b', 'c', 'd']

실습 3. 쉼표 기준으로 나누기 정답

CODE

```
s = "a,b,c,d"
print(s.split(","))    # ['a', 'b', 'c', 'd']  (쉼표 기준)
```

split maxsplit - 횟수 제한

괄호에 **횟수**를 함께 넣으면 그만큼만 나눔

'a-b-c-d'.split('-', 1)은 첫 자리만 나눔

맨 앞 항목만 분리하고 뒤는 통째로 둘 때

자주 쓰진 않는 옵션

실습 4. 나누는 횟수 제한하기

목표	split(구분자, 횟수) 로 나누는 횟수를 제한
단계	① 'a,b,c,d' 저장 → ② 첫 번째 ','을 기준으로 한 번만 나누기
예상 결과	['a', 'b,c,d']

실습 4. 나누는 횟수 제한하기 정답

CODE

```
s = "a,b,c,d"
print(s.split(",", 1))    # ['a', 'b,c,d']  (앞에서 1번만 나눔)
```

join() - 합치기

join() - 리스트의 여러 조각을 하나의 문자열로 합치기

'-'.join(parts)는 조각들 사이에 하이픈을 끼움

순서가 독특 - 구분자가 앞에 옴

split으로 나눈 데이터를 다시 합칠 때

join 구분자 지정

join 앞 **구분자**에 따라 합쳐진 모양이 달라짐

쉼표로 합치면 사과,배,감 / 공백이면 사과 배 감

빈 값('')으로 합치면 딱 붙음 (사과배감)

만들 결과에 맞게 구분자 선택

실습 5. 리스트 합치기

목표	join() 으로 리스트의 문자열들을 구분자로 이어 붙이기
단계	① ['2025','01','15'] 준비 → ② '-'으로 연결하여 합치기
예상 결과	2025-01-15

실습 5. 리스트 합치기 정답

CODE

```
parts = ["2025", "01", "15"]
print("-".join(parts))    # 2025-01-15  (사이에 - 넣어 합침)
```

print의 sep·end로 구분자 넣기

print(값, 값, sep='구분자') 로 출력할 때 값들 사이에 구분자 삽입

print('2025','01','15', sep='-') 는 2025-01-15

end 는 출력 끝 문자 지정 (기본값은 줄바꿈)

join 은 리스트를 문자열로 합침 / sep 은 출력할 때 사이에 넣는 구분자

실습 6. sep으로 이어 출력하기

목표	print 의 sep 으로 값들 사이에 구분자를 넣어 한 줄로 출력
단계	2025·1·15 를 '-' 와 ','로 총 2가지 방법을 사용해 이어서 출력
예상 결과	2025-1-15 / 2025, 1, 15

실습 6. sep으로 이어 출력하기 정답

CODE

```
print(2025, 1, 15, sep='-')     # 2025-1-15
print(2025, 1, 15, sep=', ')    # 2025, 1, 15
```

split과 join은 짝꿍

split은 문자열→리스트, join은 리스트→문자열 (정반대)

방향만 반대, 구분자라는 같은 개념 사용

둘을 묶으면 구분자를 통째로 교체 가능

문법 - 문자열.split() / 구분자.join()

split·join 짝꿍 - 코드 예시

split으로 나누고 join으로 구분자 교체

CODE

```
text = '사과,배,감'
parts = text.split(',')
joined = ' / '.join(parts)
print(joined)          # 사과 / 배 / 감
```

실습 7. 구분자 통째로 바꾸기

목표	split 으로 나눈 뒤 다른 구분자로 join 해 구분자를 통째로 교체
단계	① '2025/01/15' 저장 → ② '/'을 기준으로 나눈 후 '-' 으로 합치기
예상 결과	2025-01-15

실습 7. 구분자 통째로 바꾸기 정답

CODE

```
raw = "2025/01/15"
parts = raw.split("/")          # ['2025', '01', '15']
print("-".join(parts))          # 2025-01-15  (/ 를 - 로 교체)
```

join 활용 - 정리된 데이터 다시 합치기

나누고 정리한 조각을 join으로 **다시 합쳐 마무리**

날짜 조각을 '-'.join으로 2025-1-15 형태로

나누기(split) → 정리(strip/lower) → 합치기(join)

텍스트 정리의 기본 3단계 패턴

replace vs split 선택 기준

Q1. 언제 replace를 쓰나요?

A. 바꾸거나 지울 때 (결과는 하나의 문자열)

Q2. 언제 split을 쓰나요?

A. 나눠서 일부를 골라 쓸 때 (결과는 리스트)

Q3. 고르는 기준은?

A. 바꿔 하나로 둘지, 나눠 따로 쓸지

strip+split 함께 쓰기

strip 먼저, split 나중 순으로 깔끔하게 나누기

text.strip().split(',') 형태로 체이닝

앞뒤 공백·줄바꿈을 떼고 나눠 첫·끝 조각이 깔끔

파일에서 읽은 줄을 다룰 때 거의 항상 이 순서

split 후 각 항목 정리

나눈 **각 조각**도 strip 등으로 따로 정리 가능

'사과, 배, 감'.split(',')은 배·감 앞에 공백 남음

필요한 조각을 번호로 꺼내 strip으로 정리

구분자에 ', '처럼 공백 포함시키면 한 번에 정리

실습 8. CSV 한 줄에서 값 꺼내 정리하기

목표	split 으로 나눈 뒤 원하는 조각을 골라 strip·lower 로 정리
단계	① '1, NORMAL ,25.3' 저장 ② ',' 기준으로 나누어 1번 인덱스를 공백 제거와 소문자로 변환해 출력
예상 결과	normal

실습 8. CSV 한 줄에서 값 꺼내 정리하기 정답

CODE

```
raw = "1, NORMAL ,25.3"
parts = raw.split(",")          # ['1', ' NORMAL ', '25.3']
status = parts[1].strip().lower()
print(status)                   # normal  (공백 제거 후 소문자)
```

흔한 실수 - join 대상은 문자열 리스트

Q1. join은 무엇만 합칠 수 있나요?

A. 문자열 조각만 (숫자가 섞이면 오류)

Q2. 숫자가 섞이면?

A. str로 먼저 문자열로 변환

Q3. 오류가 나면 먼저?

A. 리스트 안에 숫자가 섞였는지 확인

치환·분리·결합 메서드 정리

나누기(split) → 정리(strip/lower/replace) → 합치기(join)

결과는 replace는 문자열, split은 리스트, join은 문자열

원본 안 바꿈 - 결과를 변수에 다시 받기

join 대상은 모두 문자열 (숫자는 str로)

치환·분리·결합 비교표

메서드	용도	결과
replace(A, B)	A를 B로 바꾸거나 제거	문자열
split()	공백으로 나누기	리스트
split(',')	구분자로 나누기	리스트
구분자.join()	조각 합치기	문자열

감사합니다.

posco